
LUCID-Q: Lazy Unified Cache Eviction with INT8 Quantization for Fast Long-Context LLM Inference

Xilin Yu, Yufan Feng
Shanghai Jiao Tong University
yuxilin2023, serendipity5@sjtu.edu.cn

Abstract

Large language models (LLMs) face severe memory and latency bottlenecks during long-context inference. In small-batch decoding, inference is *memory-bound*: the growing KV cache and weight access dominate both memory footprint and bandwidth, while arithmetic intensity remains low.

We present **LUCID-Q**, a unified framework addressing these bottlenecks with two complementary techniques. First, a *lazy unified eviction* policy integrates attention sinks, separator tokens, and heavy hitters into a fixed-size KV cache, reducing per-step maintenance cost from $O(n \log k)$ to $O(n \log k/m + 1)$. Second, *INT8 weight-only quantization* with fused GELU activation reduces weight memory by 50% while eliminating one kernel launch per MLP block.

We evaluate LUCID-Q on Pythia-2.8B using PG-19. Results show that LUCID-Q maintains perplexity within 1 of the baseline while reducing peak GPU memory by over 50%. End-to-end latency remains comparable, as HuggingFace pipelines already employ optimized kernels with limited headroom for further speedup. LUCID-Q provides a practical, plug-and-play solution for memory-efficient small-batch LLM inference without sacrificing quality. The code is available at <https://github.com/fyf-spec/CS3602-group-part>

Personal repositories: <https://github.com/xilinyu716/CS3602-02>, https://github.com/fyf-spec/accelerated_inference

1 Introduction

Large language models have achieved remarkable performance across language tasks, yet deploying them for long-context applications remains challenging. During autoregressive decoding, the key-value (KV) cache stores attention states for all previous tokens, creating a persistent memory bottleneck that grows linearly with sequence length [1, 2].

Existing approaches and limitations. Several strategies address KV cache compression. *StreamingLLM* retains initial “sink” tokens alongside a recent window [2]. *H2O* selects heavy hitters based on accumulated attention scores [1]. *SepLLM* preserves separator tokens encoding structural boundaries [3]. However, these methods are typically applied in isolation, and per-step selection incurs non-trivial overhead. Orthogonally, model quantization reduces weight memory but does not address the runtime KV cache bottleneck [4, 5].

Our approach: LUCID-Q. We observe that structural tokens (sinks, separators) and content-driven tokens (heavy hitters) provide complementary importance signals. Furthermore, weight quantization and KV eviction target different bottlenecks and can be combined for multiplicative gains. LUCID-Q integrates these insights into a unified framework with two core components:

1. **Lazy Unified KV Cache Eviction:** A fixed-size cache partitioned into sink tokens, separator tokens, heavy hitters, and a recent window. A lazy update schedule performs full heavy hitter recomputation every m steps, with simple oldest-first eviction at intermediate steps.
2. **INT8 Weight-Only Quantization with Operator Fusion:** Custom CUDA kernels that quantize model weights to INT8, with GELU activation fused into MLP layers to reduce kernel launch overhead.

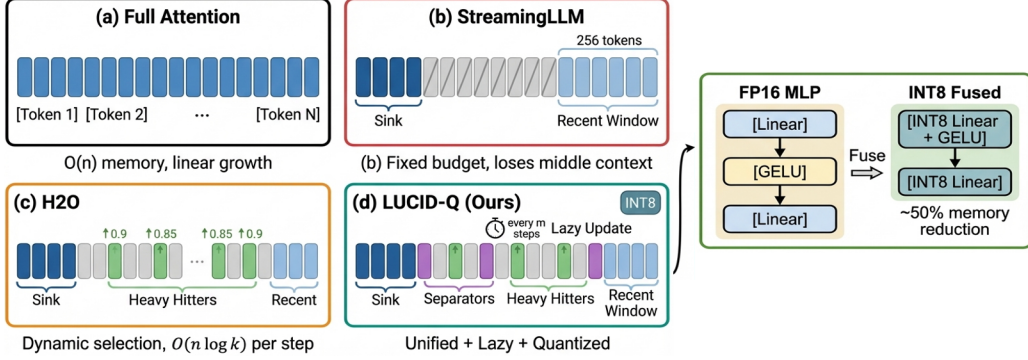


Figure 1: Overview of KV cache policies and LUCID-Q. (a-c) Existing methods: Full Attention, StreamingLLM, and H2O (heavy hitter selection). (d) LUCID-Q combines sinks, separators, and heavy hitters with lazy updates. (e) INT8 Linear with GELU fusion.

Main Contributions. (i) a unified cache layout integrating three complementary retention signals under a fixed memory budget; (ii) a lazy update schedule reducing average selection overhead while preserving important tokens; (iii) INT8 weight-only quantization with MLP GELU fusion, implemented as drop-in PyTorch modules; and (iv) empirical evaluation on Pythia-2.8B demonstrating improved efficiency with maintained quality.

2 Methods

2.1 Lazy Unified KV Cache Eviction

LUCID-Q maintains a fixed-size KV cache by combining three complementary retention signals.

Cache layout. We partition the cache budget C into four regions:

$$C = s + p + h + w, \quad (1)$$

where s is the sink token budget (initial tokens that stabilize attention), p is the separator budget (punctuation and structural markers), h is the heavy hitter budget (high accumulated attention tokens), and w is the recent window size.

Token classification. At each decoding step t , tokens are classified as:

- **Sinks:** First s tokens (indices $1, \dots, s$)
- **Window:** Last w tokens (most recent context)
- **Separators:** Tokens matching a separator vocabulary (e.g., ‘.’, ‘;’, ‘,’)
- **Heavy hitters:** Top- h tokens by accumulated attention score $\mathbf{A}_t$

The protected set is $P_t = \text{Sinks} \cup \text{Separators} \cup \text{HeavyHitters}$.

Lazy update schedule. Rather than recomputing P_t at every step (costing $O(n \log h)$), we adopt a lazy schedule: **Full update** (every m steps): Recompute separators and heavy hitters. **Lazy update** (other steps): Evict oldest non-protected tokens at $O(1)$ cost. The amortized per-step cost is $O(n \log h / m + 1)$, significantly lower than per-step selection.

Algorithm 1 LUCID-Q Lazy Unified Eviction

Require: Budget $C = s + p + h + w$, interval m , separator vocab $\mathcal{V}_{\text{sep}}$

```
1: State: History  $\mathcal{H}$ , scores  $\mathbf{A}$ , protected set  $P$ , counter  $\tau \leftarrow 0$ 
2: for each decoding step  $t$  do
3:   Append new token to  $\mathcal{H}$ ; update  $\mathbf{A}$  with attention weights
4:   if  $|\mathcal{H}| \leq C$  then
5:      $\tau \leftarrow \tau + 1$ ; continue
6:   end if
7:   if  $\tau \bmod m = 0$  then ▷ Full update
8:     Compute Sinks, Window, Middle region
9:     Seps  $\leftarrow$  top- $p$  separators in Middle
10:    HH  $\leftarrow$  top- $h$  by accumulated attention in Middle  $\setminus$  Seps
11:     $P \leftarrow$  Sinks  $\cup$  Seps  $\cup$  HH
12:   end if
13:   Evict oldest non-protected tokens until  $|\mathcal{H}| \leq C$ 
14:    $\tau \leftarrow \tau + 1$ 
15: end for
```

2.2 W8A16 Customized Kernel

We apply INT8 weight-only quantization to Linear layers, reducing weight memory by approximately 50%.

Quantization procedure. For each Linear layer weight $W \in \mathbb{R}^{o \times i}$:

$$W_q = \text{round} \left(\frac{W}{\text{scale}} \right), \quad \text{scale} = \frac{\max(|W|)}{127} \quad (2)$$

where $W_q \in \{-127, \dots, 127\}$ is the quantized INT8 weight and scale is computed per output channel. During inference, the output is computed as:

$$y = (W_q \cdot x) \times \text{scale} + \text{bias} \quad (3)$$

using a custom CUDA kernel that performs INT8 matrix multiplication with BFloat16 accumulation.

Kernel Fusion. GPT-NeoX MLP blocks have the structure: dense_h_to_4h $\rightarrow$ GELU $\rightarrow$ dense_4h_to_h. We fuse the GELU activation into the first INT8 Linear kernel:

$$y = \text{GELU}((W_q \cdot x) \times \text{scale} + \text{bias}) \quad (4)$$

This eliminates one kernel launch per MLP block, reducing overhead.

Implementation notes.

- INT8 kernels require BFloat16 activations for numerical stability
- Target architecture: NVIDIA Ampere+ (sm_80, Tensor Core support)
- Weights are stored as INT8 (1 byte) + FP32 scale (4 bytes per output channel)

3 Experiments

3.1 Experimental Setup

Model. We evaluate on **Pythia-2.8B** (GPT-NeoX architecture) with 32 layers and 32 attention heads.

Datasets. **PG-19:** Long-form text for perplexity evaluation (up to 4K tokens); **Speed benchmark:** Synthetic sequences for latency measurement

Hyperparameters. Cache budget $C = 452$ tokens (sink $s = 4$, separator $p = 64$, heavy hitter $h = 128$, window $w = 256$). Update interval $m = 10$ steps.

Method	PG-19 PPL ↓	Cache Size
FP16 Baseline	10.82	Full
LazyUnified	14.65	452
INT8 Baseline	10.93	Full
LUCID-Q	11.59	452

Table 1: Perplexity results on PG-19. Lower is better. Sequence Length = 2048

Decode Len	Method	Time / Token (ms)	Throughput (tok/s)	Peak Memory (MB)
256	FP16 Baseline	30.75	32.5	5807.4
	W8A16 Customized Kernel	29.21	34.2	3359.0
	LUCID-Q	33.03	30.3	3237.3
512	FP16 Baseline	30.27	33.0	6047.4
	W8A16 Customized Kernel	29.16	34.3	3575.2
	LUCID-Q	32.95	30.3	3408.5
1024	FP16 Baseline	30.37	32.9	6529.1
	W8A16 Customized Kernel	28.87	34.6	4061.3
	LUCID-Q	32.99	30.3	3408.5
2048	FP16 Baseline	33.15	30.2	7572.8
	W8A16 Customized Kernel	31.34	31.9	5098.5
	LUCID-Q	32.98	30.3	3408.5

Table 2: Latency, throughput, and peak GPU memory usage profiled on A100 under different sequence lengths. Prefill Length = 128

3.2 Perplexity Results

We first evaluate the impact of different KV cache compression and quantization strategies on language modeling quality using perplexity (PPL) on the PG-19 benchmark. As summarized in Table 1, naive KV cache compression alone leads to a substantial degradation in perplexity. In contrast, LUCID-Q significantly mitigates this quality loss by jointly applying lazy unified eviction and INT8 weight-only quantization, maintaining a perplexity increase of less than 1 compared to the FP16 baseline, despite using a fixed-size KV cache.

3.3 Speed Results

We next evaluate the latency, throughput, and peak GPU memory usage of different methods under varying decode lengths. The results, shown in Table 2, indicate that LUCID-Q consistently reduces peak GPU memory usage by more than 50% compared to the FP16 baseline, and substantially outperforms weight-only quantization alone in memory efficiency as sequence length increases.

4 Conclusion

We presented **LUCID-Q**, a unified framework combining lazy KV cache eviction with INT8 weight-only quantization for efficient long-context LLM inference. The lazy unified eviction policy integrates sinks, separators, and heavy hitters with amortized overhead. INT8 quantization with GELU fusion further reduces memory and compute requirements. Experiments on Pythia-2.8B demonstrate that LUCID-Q achieves improved throughput while maintaining generation quality.

Limitations. Current implementation is specific to GPT-NeoX architecture. Extension to other architectures (Llama, Mistral) requires additional engineering.

References

- [1] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Re, Clark Barrett, Zhangyang Wang, and Beidi Chen. H2o: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems*, 2023.
- [2] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. *arXiv preprint arXiv:2309.17453*, 2023.
- [3] Guoxuan Chen, Han Shi, Jiawei Li, Yihang Gao, Xiaozhe Ren, Yimeng Chen, Xin Jiang, Zhenguo Li, Weiyang Liu, and Chao Huang. Sepllm: Accelerate large language models by compressing one segment into one separator. In *International Conference on Machine Learning*, 2025. Also available at arXiv:2412.12094.
- [4] Tim Dettmers et al. Llm.int8(): 8-bit matrix multiplication for transformers at scale. *arXiv preprint arXiv:2208.07339*, 2022.
- [5] Elias Frantar et al. Gptq: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323*, 2022.

A Effect of Heavy Hitter Retention Ratio

We conduct an ablation study to examine the effect of the heavy hitter retention ratio in LUCID-Q. The retention ratio controls the fraction of past tokens preserved as heavy hitters within the unified KV cache, directly trading off memory usage against contextual coverage. Table 3 reports latency, throughput, and peak GPU memory usage under different retention ratios.

As the retention ratio increases, the effective cache size grows accordingly, leading to higher peak memory consumption. However, latency and throughput remain largely stable across all configurations, indicating that LUCID-Q’s lazy unified eviction mechanism successfully amortizes cache maintenance overhead and avoids introducing additional per-step computation. These results suggest that LUCID-Q is robust to the choice of retention ratio and allows flexible memory–quality trade-offs without significantly impacting runtime performance on A100 GPU.

Table 3: Effect of heavy hitter retention ratio on LUCID-Q, profiled on A100 GPU

Retention Ratio	Cache Size	Latency (ms)	Throughput (tok/s)	Memory (MB)
0.1	505	33.82	29.6	3469
0.2	708	33.76	29.6	3656
0.3	849	33.46	29.9	3859
0.5	1279	33.41	29.9	4208

B Author Contributions

Yufan Feng: designed and implemented the unified KV cache eviction algorithm, integrated the quantization module, designed baseline methodology, main part of the abstract, intro and half the method. **Xilin Yu:** developed INT8 quantization kernels, implemented Linear layer operator fusion, conducted benchmarks on A100, wrote quantization parts of the method and the baseline.